

Clinic Management System — RESTful API

Full Technical Planning Document (PRD + SRS + Implementation Roadmap)

Table of Contents

- 1. [Project Overview](#)
- 2. [Stakeholders & Roles](#)
- 3. [User Stories](#)
- 4. [Functional Requirements](#)
- 5. [Non-Functional Requirements](#)
- 6. [System Architecture](#)
- 7. [Database Design](#)
- 8. [API Endpoint Design](#)
- 9. [Tech Stack](#)
- 10. [Laravel Project File Structure](#)
- 11. [Business Rules](#)
- 12. [Migration Order](#)
- 13. [Development Phases & Roadmap](#)

1. Project Overview

Field	Detail
Product Name	Clinic Management System (CMS)
Type	RESTful API — Backend Only
Framework	Laravel 11
Database	MySQL 8
Auth	Laravel Sanctum (API Tokens)
Scope	Multi-clinic, multi-role platform

Business Goals

- Digitize appointment booking across multiple clinic branches
- Enable doctors to manage patient prescriptions and schedules efficiently

- Provide super admin with cross-clinic visibility and control

2. Stakeholders & Roles

Role	Description	Clinic Scope
Super Admin	System owner — monitors and manages everything	All clinics
Doctor (Admin)	Views their appointments, manages prescriptions	Own clinic only
Doctor Assistant	Registers patients, books and manages appointments	Own clinic only

Role Hierarchy (Visual)

```
Super Admin
├─ Clinic A
│   ├─ Doctor (Admin)
│   └─ Doctor Assistant
├─ Clinic B
│   ├─ Doctor (Admin)
│   └─ Doctor Assistant
└─ Clinic N ...
```

Key architectural constraint: Every doctor and assistant is scoped to exactly one clinic. A middleware layer must enforce this — no cross-clinic data leakage.

3. User Stories

3.1 Super Admin

ID	User Story	Priority
US-SA-01	As a super admin, I want to create and manage clinics so I can onboard new branches	High
US-SA-02	As a super admin, I want to create doctor and assistant accounts and assign them to a clinic	High
US-SA-03	As a super admin, I want to activate or deactivate any clinic or user account	High

ID	User Story	Priority
US-SA-04	As a super admin, I want to view all appointments across all clinics with filtering by date or clinic	High
US-SA-05	As a super admin, I want to view all prescriptions across all clinics	Medium
US-SA-06	As a super admin, I want a dashboard showing key system statistics	Medium

3.2 Doctor (Admin)

ID	User Story	Priority
US-DR-01	As a doctor, I want to define my weekly availability so assistants know when I can see patients	High
US-DR-02	As a doctor, I want to view all my upcoming and past appointments	High
US-DR-03	As a doctor, I want to view the details of any appointment including patient information	High
US-DR-04	As a doctor, I want to create a prescription linked to a specific appointment	High
US-DR-05	As a doctor, I want to update an existing prescription	High
US-DR-06	As a doctor, I want to mark an appointment as completed or cancelled	Medium
US-DR-07	As a doctor, I want to view a patient's full visit and prescription history	Medium
US-DR-08	As a doctor, I want to view the list of all patients registered in my clinic	Medium

3.3 Doctor Assistant

ID	User Story	Priority
US-AS-01	As an assistant, I want to register new patients so they can be booked for appointments	High

ID	User Story	Priority
US-AS-02	As an assistant, I want to search for existing patients by name or phone number	High
US-AS-03	As an assistant, I want to view the doctor's available time slots for a given date	High
US-AS-04	As an assistant, I want to book an appointment for a patient in an available slot	High
US-AS-05	As an assistant, I want to cancel an appointment	High
US-AS-06	As an assistant, I want to reschedule an appointment to a different available slot	Medium
US-AS-07	As an assistant, I want to view today's full appointment list	High
US-AS-08	As an assistant, I want to view a patient's previous appointment history	Medium

4. Functional Requirements

4.1 Authentication & Authorization

- **FR-AUTH-01:** The system shall authenticate users via API tokens (Laravel Sanctum)
- **FR-AUTH-02:** The system shall enforce Role-Based Access Control (RBAC) using middleware
- **FR-AUTH-03:** Users with roles `doctor` or `assistant` shall only access data from their assigned clinic
- **FR-AUTH-04:** The system shall support secure logout and token invalidation
- **FR-AUTH-05:** Passwords shall be hashed with bcrypt and never returned in any API response

4.2 Clinic Management

- **FR-CLI-01:** Super admin can create, update, and soft-delete clinics
- **FR-CLI-02:** Super admin can activate or deactivate a clinic
- **FR-CLI-03:** Super admin can view all clinics with their associated users

4.3 User Management

- **FR-USR-01:** Super admin can create user accounts with roles: `doctor` or `assistant`

- **FR-USR-02:** Super admin must assign a clinic to every non-super-admin user at creation
- **FR-USR-03:** Super admin can activate or deactivate user accounts
- **FR-USR-04:** Super admin can update user profile information

4.4 Patient Management

- **FR-PAT-01:** Assistants can create new patient records within their clinic
- **FR-PAT-02:** Assistants can search patients by name or phone number
- **FR-PAT-03:** Patient data includes: name, phone, email, date of birth, gender, address
- **FR-PAT-04:** Patients are scoped to the clinic where they were registered

4.5 Schedule Management

- **FR-SCH-01:** Doctors can define their weekly availability (day of week, start time, end time)
- **FR-SCH-02:** Doctors can specify a slot duration in minutes (e.g. 20, 30, 60)
- **FR-SCH-03:** The system shall compute and return available time slots dynamically based on the schedule minus already-booked appointments
- **FR-SCH-04:** Doctors can update or delete schedule entries

4.6 Appointment Management

- **FR-APP-01:** Assistants can book an appointment by specifying: doctor, patient, date, and start time
- **FR-APP-02:** Appointments shall have a status of: `pending`, `confirmed`, `cancelled`, or `completed`
- **FR-APP-03:** The system shall prevent double-booking of the same time slot for the same doctor
- **FR-APP-04:** Assistants can cancel appointments (status → `cancelled`)
- **FR-APP-05:** Assistants can reschedule appointments (change date/time to an available slot)
- **FR-APP-06:** Doctors can update the status of their own appointments
- **FR-APP-07:** The system shall return a list of available slots when given a doctor ID and date

4.7 Prescription Management

- **FR-PRE-01:** Doctors can create a prescription linked to a specific appointment
- **FR-PRE-02:** Each prescription must include a diagnosis and can include notes
- **FR-PRE-03:** Each prescription can have one or more items (medicine name, dosage, frequency, duration)
- **FR-PRE-04:** Doctors can update prescriptions they created
- **FR-PRE-05:** Super admin can view any prescription in the system

4.8 Dashboard & Reporting

- **FR-DSH-01:** Super admin dashboard shows: total clinics, doctors, assistants, patients, appointments
 - **FR-DSH-02:** Doctor dashboard shows: today's appointments, total upcoming, total completed
 - **FR-DSH-03:** Super admin can filter appointments by clinic, doctor, status, or date range
-

5. Non-Functional Requirements

5.1 Security

- **NFR-SEC-01:** All API traffic must use HTTPS in production
- **NFR-SEC-02:** Input validation must be enforced on all write endpoints via Laravel Form Requests
- **NFR-SEC-03:** A `ClinicScopeMiddleware` must restrict all data queries to the authenticated user's clinic
- **NFR-SEC-04:** Rate limiting must be applied to the login endpoint (max 5 attempts/minute)
- **NFR-SEC-05:** Sensitive fields (password, tokens) are never returned in any response

5.2 Performance

- **NFR-PER-01:** All list endpoints must support pagination (default: 15 items per page)
- **NFR-PER-02:** Eager loading must be used on relationships to prevent N+1 query problems
- **NFR-PER-03:** Available slots computation should be cached per doctor-per-date using Redis

5.3 Scalability

- **NFR-SCA-01:** Adding a new clinic requires only a database record — no code changes
- **NFR-SCA-02:** All clinic-specific filtering must be query-level, not application-level

5.4 Maintainability

- **NFR-MAI-01:** Business logic must live in Service classes, not controllers
- **NFR-MAI-02:** Controllers must only: receive a request → call a service → return a response
- **NFR-MAI-03:** All API responses must follow a consistent JSON envelope format
- **NFR-MAI-04:** Code must follow SOLID principles and PSR-12 coding style

5.5 Availability

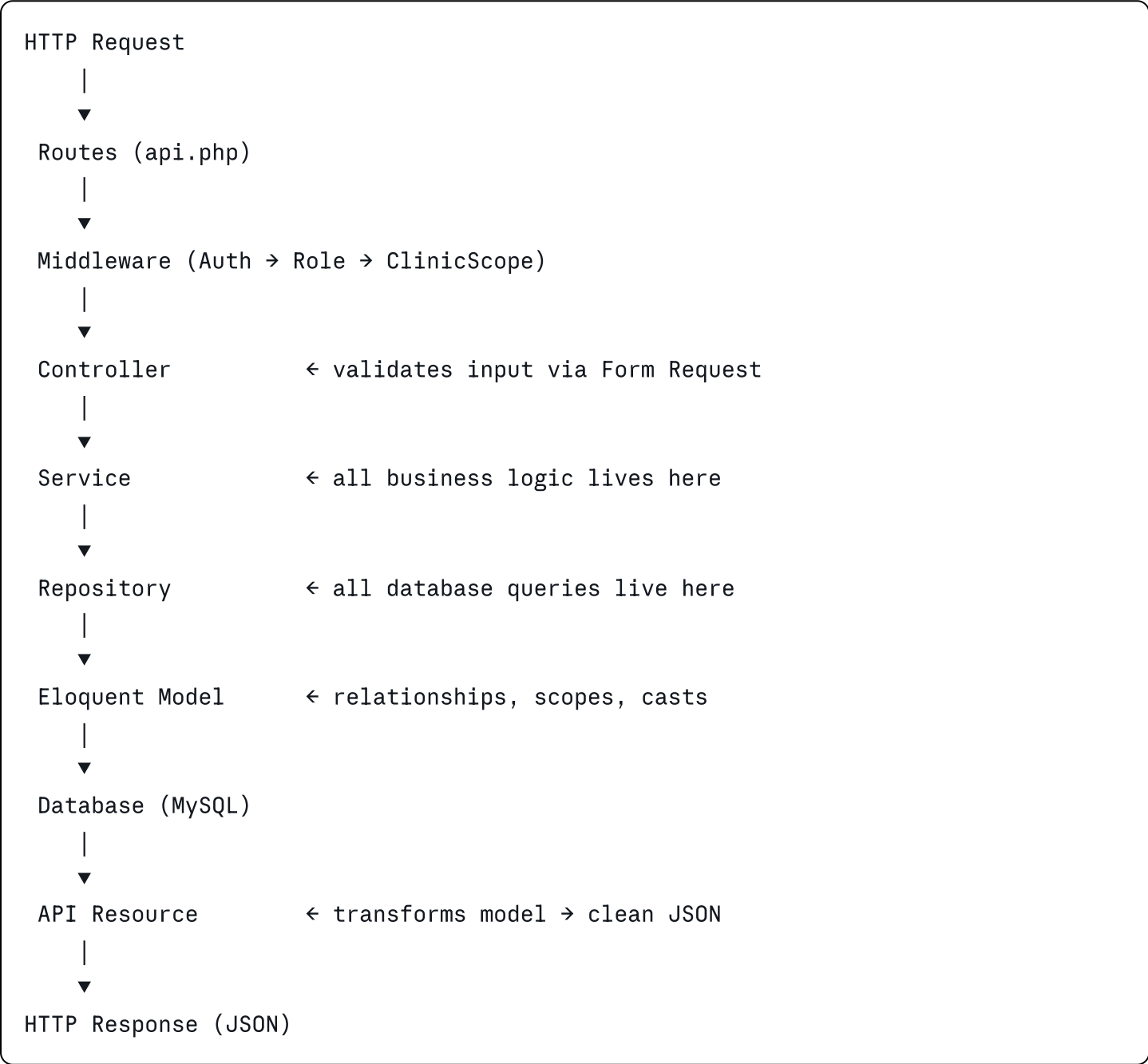
- **NFR-AVA-01:** Target 99.5% uptime
- **NFR-AVA-02:** Failed queue jobs must be automatically retried

5.6 Documentation

- **NFR-DOC-01:** All endpoints documented in a Postman Collection (exported as JSON)
- **NFR-DOC-02:** Optional: Swagger/OpenAPI via L5-Swagger package

6. System Architecture

Architecture Pattern: Layered (Controller → Service → Repository → Model)



Layer Responsibilities

Layer	Class Type	Responsibility
Routes	routes/api.php	URL mapping, middleware groups

Layer	Class Type	Responsibility
Middleware	RoleMiddleware	Block requests from wrong roles
Middleware	ClinicScopeMiddleware	Inject clinic_id into all queries
Controller	XxxController	Receive → delegate → respond. No logic.
Form Request	XxxRequest	Validation rules and error messages
Service	XxxService	Business logic, orchestrates calls to repositories
Repository	XxxRepository	Database queries only — no logic
Model	Xxx	Relationships, fillable, casts, scopes
API Resource	XxxResource	Shape the response — hide sensitive fields

7. Database Design

7.1 clinics Table

Column	Type	Notes
id	bigint PK	
name	string	
address	string	
phone	string	
email	string	
is_active	boolean	default: true
created_at	timestamp	
updated_at	timestamp	

7.2 users Table

Column	Type	Notes
id	bigint PK	

Column	Type	Notes
clinic_id	bigint FK	nullable — null means super_admin
name	string	
email	string	unique
password	string	bcrypt hashed
phone	string	
role	enum	(super_admin), (doctor), (assistant)
is_active	boolean	default: true
created_at	timestamp	
updated_at	timestamp	

7.3 (patients) Table

Column	Type	Notes
id	bigint PK	
clinic_id	bigint FK	
name	string	
email	string	nullable
phone	string	
date_of_birth	date	nullable
gender	enum	(male), (female)
address	text	nullable
created_at	timestamp	
updated_at	timestamp	

7.4 (doctor_schedules) Table

Column	Type	Notes
id	bigint PK	

Column	Type	Notes
doctor_id	bigint FK	→ users (role: doctor)
clinic_id	bigint FK	
day_of_week	tinyint	0 = Sunday ... 6 = Saturday
start_time	time	
end_time	time	
slot_duration	tinyint	minutes — default: 30
is_active	boolean	
created_at	timestamp	
updated_at	timestamp	

7.5 `appointments` Table

Column	Type	Notes
id	bigint PK	
clinic_id	bigint FK	
doctor_id	bigint FK	→ users
patient_id	bigint FK	→ patients
booked_by	bigint FK	→ users (the assistant who booked)
appointment_date	date	
start_time	time	
end_time	time	
status	enum	<code>pending</code> , <code>confirmed</code> , <code>cancelled</code> , <code>completed</code>
notes	text	nullable
created_at	timestamp	
updated_at	timestamp	

Unique constraint: `(doctor_id, appointment_date, start_time)` — prevents double-booking at the database level.

7.6 `prescriptions` Table

Column	Type	Notes
id	bigint PK	
appointment_id	bigint FK	
doctor_id	bigint FK	→ users
patient_id	bigint FK	→ patients
clinic_id	bigint FK	
diagnosis	text	
notes	text	nullable
created_at	timestamp	
updated_at	timestamp	

7.7 `prescription_items` Table

Column	Type	Notes
id	bigint PK	
prescription_id	bigint FK	
medicine_name	string	
dosage	string	e.g. "500mg"
frequency	string	e.g. "twice daily"
duration	string	e.g. "7 days"
notes	text	nullable
created_at	timestamp	
updated_at	timestamp	

7.8 Entity Relationship Summary

```
Clinic —< User (doctor, assistant)
Clinic —< Patient
Clinic —< Appointment
Clinic —< Prescription
User (doctor) —< DoctorSchedule
User (doctor) —< Appointment (as doctor_id)
User (assistant) —< Appointment (as booked_by)
Patient —< Appointment
Appointment —1 Prescription
Prescription —< PrescriptionItem
```

8. API Endpoint Design

8.1 Standard Response Envelope

Success Response:

```
json
{
  "success": true,
  "message": "Appointments retrieved successfully",
  "data": { },
  "meta": {
    "current_page": 1,
    "last_page": 5,
    "per_page": 15,
    "total": 72
  }
}
```

Error Response:

```
json
{
  "success": false,
  "message": "Validation failed",
  "errors": {
    "email": ["The email field is required."],
    "role": ["The selected role is invalid."]
  }
}
```

8.2 Authentication Endpoints

Method	Endpoint	Access	Description
POST	<code>/api/auth/login</code>	Public	Login, get token
POST	<code>/api/auth/logout</code>	All	Revoke token
GET	<code>/api/auth/me</code>	All	Get current user info

8.3 Super Admin — Clinics

Method	Endpoint	Description
GET	<code>/api/super-admin/clinics</code>	List all clinics (paginated)
POST	<code>/api/super-admin/clinics</code>	Create a new clinic
GET	<code>/api/super-admin/clinics/{id}</code>	View clinic detail
PUT	<code>/api/super-admin/clinics/{id}</code>	Update clinic info
PATCH	<code>/api/super-admin/clinics/{id}/toggle</code>	Activate / Deactivate

8.4 Super Admin — Users

Method	Endpoint	Description
GET	<code>/api/super-admin/users</code>	List all users (filterable by role)
POST	<code>/api/super-admin/users</code>	Create doctor or assistant account
GET	<code>/api/super-admin/users/{id}</code>	View user detail
PUT	<code>/api/super-admin/users/{id}</code>	Update user profile
PATCH	<code>/api/super-admin/users/{id}/toggle</code>	Activate / Deactivate

8.5 Super Admin — Monitoring

Method	Endpoint	Description
GET	<code>/api/super-admin/dashboard</code>	System-wide stats

Method	Endpoint	Description
GET	<code>/api/super-admin/appointments</code>	All appointments (filter: clinic, doctor, status, date)
GET	<code>/api/super-admin/prescriptions</code>	All prescriptions (paginated)

8.6 Doctor — Schedule

Method	Endpoint	Description
GET	<code>/api/doctor/schedules</code>	View my schedule
POST	<code>/api/doctor/schedules</code>	Create a schedule entry
PUT	<code>/api/doctor/schedules/{id}</code>	Update a schedule entry
DELETE	<code>/api/doctor/schedules/{id}</code>	Delete a schedule entry

8.7 Doctor — Appointments

Method	Endpoint	Description
GET	<code>/api/doctor/appointments</code>	My appointments (filter: date, status)
GET	<code>/api/doctor/appointments/{id}</code>	Appointment detail + patient info
PATCH	<code>/api/doctor/appointments/{id}/status</code>	Update appointment status

8.8 Doctor — Prescriptions

Method	Endpoint	Description
GET	<code>/api/doctor/prescriptions</code>	My prescriptions (paginated)
POST	<code>/api/doctor/prescriptions</code>	Create prescription for appointment
GET	<code>/api/doctor/prescriptions/{id}</code>	View prescription + items
PUT	<code>/api/doctor/prescriptions/{id}</code>	Update prescription

8.9 Doctor — Patients

Method	Endpoint	Description
GET	<code>/api/doctor/patients</code>	My clinic's patients list
GET	<code>/api/doctor/patients/{id}</code>	Patient detail + appointment history

8.10 Assistant — Patients

Method	Endpoint	Description
GET	<code>/api/assistant/patients</code>	Search / list clinic patients
POST	<code>/api/assistant/patients</code>	Register a new patient
GET	<code>/api/assistant/patients/{id}</code>	Patient detail
PUT	<code>/api/assistant/patients/{id}</code>	Update patient info

8.11 Assistant — Appointments

Method	Endpoint	Description
GET	<code>/api/assistant/appointments</code>	Today's appointments for my clinic
POST	<code>/api/assistant/appointments</code>	Book a new appointment
GET	<code>/api/assistant/appointments/{id}</code>	Appointment detail
PUT	<code>/api/assistant/appointments/{id}</code>	Reschedule appointment
DELETE	<code>/api/assistant/appointments/{id}</code>	Cancel appointment
GET	<code>/api/assistant/available-slots</code>	Available slots (query: doctor_id, date)

9. Tech Stack

Concern	Tool / Package
Language	PHP 8.2+

Concern	Tool / Package
Framework	Laravel 11
Authentication	Laravel Sanctum
Database	MySQL 8
Cache & Queues	Redis
API Resources	Laravel API Resources (built-in)
Validation	Laravel Form Requests (built-in)
Testing	PHPUnit + Laravel HTTP Tests
API Docs	Postman Collection (required) + L5-Swagger (optional)
Code Style	Pint (Laravel's PSR-12 formatter)
Version Control	Git — branching: <code>main</code> , <code>dev</code> , <code>feature/*</code>

10. Laravel Project File Structure

```

clinic-api/
├── app/
│   ├── Enums/
│   │   ├── UserRole.php           ← super_admin, doctor, assistant
│   │   ├── AppointmentStatus.php ← pending, confirmed, cancelled, completed
│   │   └── Gender.php             ← male, female
│   │
│   ├── Http/
│   │   ├── Controllers/
│   │   │   ├── Auth/
│   │   │   │   └── AuthController.php
│   │   │   ├── SuperAdmin/
│   │   │   │   ├── ClinicController.php
│   │   │   │   ├── UserController.php
│   │   │   │   └── DashboardController.php
│   │   │   ├── Doctor/
│   │   │   │   ├── AppointmentController.php
│   │   │   │   ├── ScheduleController.php
│   │   │   │   ├── PrescriptionController.php
│   │   │   │   └── PatientController.php
│   │   │   └── Assistant/
│   │   │       └── PatientController.php

```

```
| | | └─ AppointmentController.php
| | |
| | └─ Middleware/
| | | └─ RoleMiddleware.php          ← checks user role
| | | └─ ClinicScopeMiddleware.php  ← injects clinic_id into request
| | |
| | └─ Requests/
| | | └─ Auth/
| | | | └─ LoginRequest.php
| | | └─ Clinic/
| | | | └─ StoreClinicRequest.php
| | | | └─ UpdateClinicRequest.php
| | | └─ User/
| | | | └─ StoreUserRequest.php
| | | | └─ UpdateUserRequest.php
| | | └─ Patient/
| | | | └─ StorePatientRequest.php
| | | | └─ UpdatePatientRequest.php
| | | └─ Schedule/
| | | | └─ StoreScheduleRequest.php
| | | └─ Appointment/
| | | | └─ StoreAppointmentRequest.php
| | | | └─ UpdateAppointmentRequest.php
| | | └─ Prescription/
| | | | └─ StorePrescriptionRequest.php
| | | | └─ UpdatePrescriptionRequest.php
| |
| └─ Models/
| | └─ User.php
| | └─ Clinic.php
| | └─ Patient.php
| | └─ DoctorSchedule.php
| | └─ Appointment.php
| | └─ Prescription.php
| | └─ PrescriptionItem.php
| |
| └─ Services/
| | └─ AuthService.php
| | └─ ClinicService.php
| | └─ UserService.php
| | └─ PatientService.php
| | └─ ScheduleService.php
| | └─ AppointmentService.php  ← contains slot availability logic
| | └─ PrescriptionService.php
| |
| └─ Repositories/
| | └─ ClinicRepository.php
```

```
| | | └─ UserRepository.php
| | | └─ PatientRepository.php
| | | └─ AppointmentRepository.php
| | | └─ ScheduleRepository.php
| | | └─ PrescriptionRepository.php
| | |
| | └─ Resources/
| |   └─ UserResource.php
| |   └─ ClinicResource.php
| |   └─ PatientResource.php
| |   └─ AppointmentResource.php
| |   └─ PrescriptionResource.php
| |   └─ PrescriptionItemResource.php
| |
| └─ database/
|   └─ migrations/
|     └─ 2024_01_01_000000_create_clinics_table.php
|     └─ 2024_01_01_000001_create_users_table.php
|     └─ 2024_01_01_000002_create_patients_table.php
|     └─ 2024_01_01_000003_create_doctor_schedules_table.php
|     └─ 2024_01_01_000004_create_appointments_table.php
|     └─ 2024_01_01_000005_create_prescriptions_table.php
|     └─ 2024_01_01_000006_create_prescription_items_table.php
|   └─ seeders/
|     └─ DatabaseSeeder.php
|     └─ SuperAdminSeeder.php
|     └─ ClinicSeeder.php
|     └─ DoctorAndAssistantSeeder.php
|
└─ routes/
  └─ api.php
```

11. Business Rules

These are constraints that must be enforced in the Service layer:

ID	Rule
BR-01	A <code>doctor</code> or <code>assistant</code> can only access data from their own clinic
BR-02	<code>super_admin</code> has no <code>clinic_id</code> and can access all data
BR-03	An appointment can only be booked on a slot where the doctor has a schedule and no other booking exists
BR-04	A prescription can only be created by the doctor assigned to that appointment
BR-05	A <code>completed</code> appointment cannot be rescheduled
BR-06	A <code>cancelled</code> appointment cannot receive a prescription
BR-07	A patient belongs to one clinic — they cannot be booked at a different clinic
BR-08	An assistant can only create patients and book appointments within their own clinic
BR-09	Double-booking must be blocked at both the application level and the database level (unique constraint)

12. Migration Order

Migrations must be run in this order to satisfy foreign key constraints:

```

Step 1: create_clinics_table
Step 2: create_users_table      (FK → clinics)
Step 3: create_patients_table   (FK → clinics)
Step 4: create_doctor_schedules_table (FK → users, clinics)
Step 5: create_appointments_table (FK → clinics, users, patients)
Step 6: create_prescriptions_table (FK → appointments, users, patients, clinics)
Step 7: create_prescription_items_table (FK → prescriptions)

```

13. Development Phases & Roadmap

Phase 1 — Project Initialization (Days 1-2)

- ☐ `laravel new clinic-api --api`
- ☐ Configure `.env` (DB, Redis, queue driver)
- ☐ Install packages: `laravel/sanctum`
- ☐ Initialize Git repository
- ☐ Set up branching strategy: `main` → `dev` → `feature/*`
- ☐ Set up Pint for code formatting

Phase 2 — Database & Models (Days 3-5)

- ☐ Write all 7 migrations in the correct order
- ☐ Create Enums: `UserRole`, `AppointmentStatus`, `Gender`
- ☐ Create all Eloquent Models with:
 - `$fillable` arrays
 - Relationships (`belongsTo`, `hasMany`, `hasOne`)
 - Attribute casts (dates, enums)
- ☐ Create and run Seeders:
 - `SuperAdminSeeder` — creates one super_admin user
 - `ClinicSeeder` — creates 2 sample clinics
 - `DoctorAndAssistantSeeder` — creates one doctor + one assistant per clinic

Phase 3 — Authentication (Days 6-7)

- ☐ Install and configure Laravel Sanctum
- ☐ Build `AuthController` with: `login`, `logout`, `me`
- ☐ Register `RoleMiddleware` in `bootstrap/app.php`
- ☐ Register `ClinicScopeMiddleware`
- ☐ Write `LoginRequest` with validation
- ☐ Test all auth endpoints in Postman

Phase 4 — Super Admin Module (Days 8-11)

- ☐ `ClinicRepository` → `ClinicService` → `ClinicController`
- ☐ `UserRepository` → `UserService` → `UserController`
- ☐ `DashboardController` with DB aggregate queries
- ☐ Create `StoreClinicRequest`, `StoreUserRequest` Form Requests
- ☐ Create `ClinicResource`, `UserResource` API Resources
- ☐ Apply `role:super_admin` middleware to all routes
- ☐ Test all super admin endpoints in Postman

Phase 5 — Doctor Module (Days 12-15)

- ☐ `ScheduleRepository` → `ScheduleService` → `ScheduleController`
- ☐ Build the available-slot computation logic in `ScheduleService`
- ☐ `AppointmentController` (doctor view) — view, update status
- ☐ `PrescriptionRepository` → `PrescriptionService` → `PrescriptionController`
- ☐ Enforce BR-04 (only the appointment's doctor can create a prescription)
- ☐ Enforce BR-06 (no prescription on cancelled appointment)
- ☐ Create all doctor Form Requests and API Resources

Phase 6 — Assistant Module (Days 16-19)

- ☐ `PatientRepository` → `PatientService` → `PatientController` (assistant)
- ☐ `AppointmentService` — book, reschedule, cancel
- ☐ Build `GET /api/assistant/available-slots` endpoint
- ☐ Enforce BR-03 (slot availability check before booking)
- ☐ Enforce BR-09 (database-level unique constraint)
- ☐ Enforce BR-07 (patient clinic match)

Phase 7 — Response Standardization (Days 20-21)

- ☐ Create a `ApiResponse` helper class or trait
- ☐ Audit every controller to use the standard response format
- ☐ Add pagination to all list endpoints
- ☐ Add query filters where required (date, status, clinic)

Phase 8 — Testing (Days 22-26)

- ☐ Write Feature Tests for Auth
- ☐ Write Feature Tests for each Super Admin endpoint
- ☐ Write Feature Tests for Doctor endpoints
- ☐ Write Feature Tests for Assistant endpoints
- ☐ Test RBAC: confirm role isolation works (doctor cannot hit assistant routes, etc.)
- ☐ Test clinic-scope isolation: confirm cross-clinic data access is blocked
- ☐ Test double-booking prevention

Phase 9 — Documentation & Final Polish (Days 27-30)

- ☐ Export Postman Collection with all endpoints, example request bodies, and responses
 - ☐ Write a `README.md` with: setup steps, env variables, how to seed, how to run tests
 - ☐ Run `php artisan pint` to format all code
 - ☐ Final code review: remove debug code, check N+1 with Laravel Debugbar
 - ☐ Optional: set up L5-Swagger for Swagger UI
-

Appendix A — Sample `routes/api.php` Structure

```
php

Route::prefix('auth')->group(function () {
    Route::post('login', [AuthController::class, 'login']);
    Route::middleware('auth:sanctum')->group(function () {
        Route::post('logout', [AuthController::class, 'logout']);
        Route::get('me', [AuthController::class, 'me']);
    });
});

Route::middleware(['auth:sanctum', 'role:super_admin'])->prefix('super-admin')->group(function () {
    Route::apiResource('clinics', ClinicController::class);
    Route::patch('clinics/{clinic}/toggle', [ClinicController::class, 'toggle']);
    Route::apiResource('users', UserController::class);
    Route::patch('users/{user}/toggle', [UserController::class, 'toggle']);
    Route::get('dashboard', [DashboardController::class, 'index']);
    Route::get('appointments', [DashboardController::class, 'appointments']);
    Route::get('prescriptions', [DashboardController::class, 'prescriptions']);
});

Route::middleware(['auth:sanctum', 'role:doctor', 'clinic.scope'])->prefix('doctor')->group(function () {
    Route::apiResource('schedules', ScheduleController::class);
    Route::get('appointments', [AppointmentController::class, 'index']);
    Route::get('appointments/{appointment}', [AppointmentController::class, 'show']);
    Route::patch('appointments/{appointment}/status', [AppointmentController::class, 'toggleStatus']);
    Route::apiResource('prescriptions', PrescriptionController::class)->except(['destroy']);
    Route::get('patients', [PatientController::class, 'index']);
    Route::get('patients/{patient}', [PatientController::class, 'show']);
});

Route::middleware(['auth:sanctum', 'role:assistant', 'clinic.scope'])->prefix('assistant')->group(function () {
    Route::apiResource('patients', PatientController::class)->except(['destroy']);
    Route::get('appointments', [AppointmentController::class, 'index']);
    Route::post('appointments', [AppointmentController::class, 'store']);
    Route::get('appointments/{appointment}', [AppointmentController::class, 'show']);
    Route::put('appointments/{appointment}', [AppointmentController::class, 'update']);
    Route::delete('appointments/{appointment}', [AppointmentController::class, 'destroy']);
    Route::get('available-slots', [AppointmentController::class, 'availableSlots']);
});
```

Appendix B — Sample `ClinicScopeMiddleware`

```
php

<?php

namespace App\Http\Middleware;

use Closure;
use Illuminate\Http\Request;

class ClinicScopeMiddleware
{
    public function handle(Request $request, Closure $next)
    {
        $user = $request->user();

        // Super admin bypasses clinic scope
        if ($user->role->value === 'super_admin') {
            return $next($request);
        }

        // Inject clinic_id into all requests
        $request->merge(['clinic_id' => $user->clinic_id]);

        return $next($request);
    }
}
```

Document Version: 1.0 / Maintained by Technical Lead / Review after each sprint.